

```

1
2 /**
3   TMR1 Generated Driver API Source File
4
5   @Company
6     Microchip Technology Inc.
7
8   @File Name
9     tmr1.c
10
11  @Summary
12    This is the generated source file for the TMR1 driver using PIC24 / dsPIC33 /
13    PIC32MM MCUs
14
15  @Description
16    This source file provides APIs for driver for TMR1.
17    Generation Information :
18      Product Revision   : PIC24 / dsPIC33 / PIC32MM MCUs - 1.75.1
19      Device             : PIC32MM0064GPL020
20    The generated drivers are tested against the following:
21      Compiler           : XC32 v2.10
22      MPLAB              : MPLAB X v5.05
23  */
24  /*
25    (c) 2016 Microchip Technology Inc. and its subsidiaries. You may use this
26    software and any derivatives exclusively with Microchip products.
27
28    THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
29    EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
30    WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
31    PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
32    WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
33
34    IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
35    INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
36    WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
37    BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE
38    FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS IN
39    ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY,
40    THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
41
42    MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
43    TERMS.
44  */
45
46  /**
47    Section: Included Files
48  */
49
50  #include <xc.h>
51  #include "tmr1.h"
52  #include "../DeElectronique.h"
53  #include "../Accelerometre.h"
54  /**
55    Section: Data Type Definitions
56  */
57
58  /** TMR Driver Hardware Instance Object
59
60    @Summary
61      Defines the object required for the maintenance of the hardware instance.
62
63    @Description
64      This defines the object required for the maintenance of the hardware
65      instance. This object exists once per hardware instance of the peripheral.
66
67    Remarks:
68      None.
69  */
70
71  typedef struct _TMR_OBJ_STRUCT
72  {

```

```

73     /* Timer Elapsed */
74     bool                                     timerElapsed;
75     /*Software Counter value*/
76     uint8_t                                 count;
77
78 } TMR_OBJ;
79
80 static TMR_OBJ tmr1_obj;
81
82 /**
83  Section: Driver Interface
84  */
85
86 void TMR1_Initialize (void)
87 {
88     uint32_t tcon_value = 0x00000000;
89     // TCKPS 1:64; TWDIS disabled; TCS PBCLK; TECS SOSC; SIDL disabled; TSYNC
    disabled; TWIP disabled; TGATE disabled; ON enabled;
90     T1CON = 0x8020;
91     tcon_value = 0x8020; // Temporary storage of value
92     T1CONCLR = _T1CON_ON_MASK; // disable Timer, before loading the period/counter
    value
93     // Period = 0.1 s; Frequency = 8000000 Hz; PR1 12500;
94     PR1 = 0x30D4;
95
96     T1CON = tcon_value; //restore the TCON value
97     IFS0CLR= 1 << _IFS0_T1IF_POSITION;
98     IEC0bits.T1IE = false;
99     tmr1_obj.timerElapsed = false;
100
101 }
102
103
104
105 void __attribute__ ((vector(_TIMER_1_VECTOR), interrupt(IPL1SOFT))) TMR1_ISR()
106 {
107     /* Check if the Timer Interrupt/Status is set */
108
109     /***User Area Begin
110
111     // ticker function call;
112     // ticker is 1 -> Callback function gets called everytime this ISR executes
113     TMR1_CallBack();
114
115     /***User Area End
116
117     tmr1_obj.count++;
118     tmr1_obj.timerElapsed = true;
119     IFS0CLR= 1 << _IFS0_T1IF_POSITION;
120 }
121
122
123 void TMR1_Period16BitSet( uint16_t value )
124 {
125     /* Update the counter values */
126     PR1 = value;
127     /* Reset the status information */
128     tmr1_obj.timerElapsed = false;
129 }
130
131 uint16_t TMR1_Period16BitGet( void )
132 {
133     return( PR1 );
134 }
135
136 void TMR1_Counter16BitSet ( uint16_t value )
137 {
138     /* Update the counter values */
139     TMR1 = value;
140     /* Reset the status information */
141     tmr1_obj.timerElapsed = false;
142 }
143

```

```

144 uint16_t TMR1_Counter16BitGet( void )
145 {
146     return( TMR1 );
147 }
148
149 // timer 1 ttes les 100 ms
150 void __attribute__ ((weak)) TMR1_CallBack(void)
151 {
152     // Add your custom callback code here
153     static uint8_t Cnt10sec = 0;
154     if (Cnt10sec < 50)
155     {
156         NUM5();
157         Cnt10sec++;
158     } else
159     {
160         NONUM();
161         MC3419_ReadIntStatusRegisterAndAck();
162         Cnt10sec = 0;
163         TMR1_Stop();
164
165         HOLD = 0;
166         while(1);
167     }
168 }
169
170
171 void TMR1_Start( void )
172 {
173     /* Reset the status information */
174     tmr1_obj.timerElapsed = false;
175
176     /*Enable the interrupt*/
177     IEC0bits.T1IE = true;
178
179     /* Start the Timer */
180     T1CONSET = _T1CON_ON_MASK;
181 }
182
183 void TMR1_Stop( void )
184 {
185     /* Stop the Timer */
186     T1CONCLR = _T1CON_ON_MASK;
187
188     /*Disable the interrupt*/
189     IEC0bits.T1IE = false;
190 }
191
192 bool TMR1_GetElapsedThenClear(void)
193 {
194     bool status;
195
196     status = tmr1_obj.timerElapsed;
197
198     if(status == true)
199     {
200         tmr1_obj.timerElapsed = false;
201     }
202     return status;
203 }
204
205 int TMR1_SoftwareCounterGet(void)
206 {
207     return tmr1_obj.count;
208 }
209
210 void TMR1_SoftwareCounterClear(void)
211 {
212     tmr1_obj.count = 0;
213 }
214
215 /**
216 End of File

```

